Resolution and machine cache (L3)

TBK SPEC 05:2026, Version 1.0

The Tebako Project

Unrestricted
November 01, 2026
TBK SPEC 05:2026, Version 1.0
Tebako
© 2026 The Tebako Project. All rights reserved

Ribose

Contents

Foreword 4

1. runtime_ref (the resolution hint in the v1 trailer) 4

2. Release index (runtime factory releases) 5

3. Machine cache layout (`~/.tebako`) 7

3.0a The manifest mirrors and the extension-slice scan 8

3.1 Home discovery 8

4. Install and trust rules (locked) 9

4a. Run-time payload resolution + lazy seeding (tebako#460, locked 2026-08-25 — the fat/lean unification) 10

5. Resolution chains 11

6. The fetch pipeline (parallel pulls, streaming integrity — locked 2026-09-18) 14

© 2026 The Tebako Project

All rights reserved. Unless otherwise specified, no part of this publication may be reproduced or utilized otherwise in any form or by any means, electronic or mechanical, including photocopying, or posting on the internet or an intranet, without prior written permission. Permission can be requested from the address below.

The Tebako Project

Suite 1, 8/F, New Henry House
10 Ice House Street
Central
Hong Kong

copyright@ribose.com
www.ribose.com

© 2026 The Tebako Project

Foreword

Normative specification of runtime/payload resolution and the shared machine cache. Status: SHIPPED for runtimes (M6–M8); payload cache PARTIAL (roadmap 07).

1. runtime_ref (the resolution hint in the v1 trailer)

FIGURE 1

```
<lang>@<lang-version>;tebako=<tebako-version>[;image][;
sha256=<64 hex>]
```

- Language-generic by construction; `ruby@...` is today's instance.
- `;sha256=` — fat payload checksum (the embedded runtime slot).
- `;image` — bare flag: image-era runtime (interpreter + separate `.tfs`). The image's expected sha256 comes from the release index — the same trust source as the executable's own checksum. Bare-flag form keeps image-era fat refs inside the 127-byte field budget.
- Refs without `;image` behave byte-identically to v1: no image lookup, no download, no env.

© 2026 The Tebako Project

2. Release index (runtime factory releases)

The index is **per-package first** (tebako#493): the resolver's preference order is the per-package shard → the derived monolith → the line index. The fallbacks stay forever — pre-shard releases are immutable and remain installable (invariant 7); a missing, unreadable, or triple-mismatched shard falls through to the next form with the URL recorded for the failure message. Post-2026-09-12 release lines ship no monolith at all (spec 13 §2a's de-rendezvous, roadmap 85): on them the shard-first order never misses, and the monolith + line-index forms exist only for the immutable pre-de-rendezvous releases. Range enumeration (choosing the newest COMPATIBLE version line) flows through the factory's in-repo L3 registry (spec 04 §2), never through a release monolith.

- `<stem>.manifest.json` — the **per-package shard**, the sidecar-era authority. The stem is the exe asset's own name (`tebako-runtime-<ver>-<lang>-<lv>-<triplet>` — suffix-less, §2's spelling rule below, on windows too). The shard is ONE manifest-entry object (the same fields the monolith's array items carry: the identity triple, `filename`, `sha256`, the additive `image` / `dll` / contract-set keys). The resolver derives the stem from the requested triple, fetches exactly one small object, and reads the contract gate from it (normalized to the array shape at the resolver boundary — tebako-resolve owns the release-card reader semantics).
- `manifest.json` — derived monolith; the same per-asset entries as an array, plus the additive image-era key `image: {filename, sha256, size_bytes}` and the additive `contract_version` key (bootstrap↔runtime contract, spec 06 §6).
- **The entry's `filename` is the ONLY authoritative asset spelling** (spec 00 §10 SSOT; tebako#456). A consumer matches the entry by the identity triple (`tebako_version`, `<lang>_version`, `platform`) — never by a synthesized name — and flows `filename` / `image.filename` / `dll.filename+install_as` verbatim into the download URL, the cache layout, and the pre-download contract gate. The `tebako-runtime-<ver>-<lang>-<lv>-<triplet>[.exe]` grammar is the factory's to declare, not the consumer's to derive (the factory publishes the windows exe SUFFIX-LESS). When no entry matches the identity triple (a pre-identity index), the synthesized spelling remains the fallback for the SHA256SUMS-only era, and the missing-entry refusal names the identity triple — the mis-lookup is diagnosed as what it is, never as a contract refusal of a hand-invented name.
- **The grammar carries the language segment** (tebako#716): `tebako-runtime-<ver>-<lang>-<lv>-<triplet>` — `<ver>` the tebako line, `<lv>` the language version, and `<lang>` the runtime's **distribution identity**: the registry row's `implementation` when the factory ships several flavors of one engine (openjdk publishes `temurin` and `graalvm` rows side by side in the same release), else the engine name (`ruby`, `python`, ...). The segment is dash-free `[a-z0-9]+` — a tebako line never carries `-`, so the dash is the unambiguous field separator. A flat directory holding several languages' runtimes is self-describing, and a cross-language version overlap can never collide byte-for-byte. **Era law:** assets published before this amendment (the ≤ v0.16.32 line) spell `tebako-runtime-<ver>-<lv>-<triplet>` — immutable, sha256-pinned by live registries and shipped packages, keep-forever (invariant 7). The identity-triple match + verbatim

© 2026 The Tebako Project

`filename` flow means consumers never branch on era for FLOWED names; a registry pick likewise flows the row's declared artifact stem into the shard probe verbatim, so the probe is exact in either era and never recomposes the identity. The few places a consumer synthesizes a name behave as follows: a REMOTE probe whose target's era is unknowable (the shard fetch on a registry-less channel, a bundle-sidecar existence probe) tries the new-era spelling first and falls back to the old-era, journaling both; the OCI lane (spec 38) opened after this amendment, so its selector-less default tag composes the new-era spelling only; the INDEX-LESS cache fallback (fat/carried installs, pre-identity manifests — all pre-#716-shaped by construction) composes the old-era spelling permanently.

- `SHA256SUMS.txt` — line-index fallback (`<sha> <file>`), carries the `<asset>.tfs` lines in the image era.
- Every payload asset also carries its own `<asset>.sha256` sidecar (coreutils `<sha> <file>` — the store marker's exact shape). The sidecar is the per-asset pin; the shard's sha fields are re-anchored to the served bytes at publish time.
- Bundle-era lines (additive, 2026-09-24, spec 36): a shard carrying the additive `bundle` block is consumed as ONE fetch — the `<stem>.tar.gz` bundle + its sidecar (`+ .asc` when declared) — verified, then unpacked in-process into the §3 store layout with every member checked against the shard's per-member pins (`InvalidBundle`, spec 36 §7). A shard without `bundle` takes the per-file path, forever. A pre-bundle resolver on a bundle-era release fails LOUD at the exe fetch (the asset does not exist — exit-69 class, naming the identity triple), never an exec of wrong bytes.
- The download base is PER-ENGINE (the chain below is SHIPPED in tebako-shim's dispatch-time fetch — tebako#567; the CLI's press-time resolver and the bootstrap consult `TEBAKO_RUNTIME_MIRROR` + the default base only). First hit wins, and every download journals the base AND the channel that supplied it:
  - **a.** `runtimes: {<engine>: {source: <base>}}` — the authored config pin (spec 07 §0; most specific: an operator who pins an engine's source means exactly that base for that engine).
  - **b.** `TEBAKO_RUNTIME_MIRROR` (https or file://) — the operator's global override, applying to every engine WITHOUT a `source:` pin. When a pin shadows a differing mirror value the shadowing is journaled, loud — never silent precedence.
  - **c.** Registry-derived — a registered registry (spec 04 §2) carrying a `kind: runtime` entry whose `engine:` (+ `implementation:` when the edge names one) matches, with a version satisfying the edge's constraint: the base derives from that version's `release.ref` — or, when the host's row carries the additive per-row `release.ref` (registry schema MINOR 6, tebako#711), from THAT ref: a version line unioning rows from several per-platform shard tags cannot name the serving tag at the version level, so the row carries it. This is the zero-config path — a third-party runtime becomes resolvable from `tebako add-registry <its feedstock>` alone, no authored config required.
  - **d.** The product default base `https://github.com/tamatebako/tebako-runtime-ruby/releases/download` (the ruby factory line — the back-compat floor every pre-chain consumer already speaks).

© 2026 The Tebako Project

An edge no channel answers is a NAMED error enumerating the channels tried (never a silent query of a base that hosts no such engine — a wrong-line 404 is diagnosed as what it is).

- Signing of the index and the artifacts it names: spec 09 §5 (every consumable index form signed — the same-channel-MITM gap closes) and spec 09 §4's runtime-fetch verification point (the per-artifact `signature` check, the unsigned-line semantics, the exit codes). The fetch journal records the verification strength alongside the base and the channel that supplied it.

3. Machine cache layout (~/.tebako)

FIGURE 2

```
runtimes/<lang>-<lv>-<ver>-<triplet>/
  tebako-runtime-<ver>-<lang>-<lv>-<triplet>[.exe]   #
interpreter (0755)
  sha256 / origin                                    #
executable markers
  tebako-runtime-<ver>-<lang>-<lv>-<triplet>.tfs     # image-
era runtime (0444)
  <image>.sha256                                     # trust
anchor ("<sha>  <file>\n")
  <image>.origin                                     #
source URL
payloads/<name>/<version>.tfs                 # registry
payloads (0444)
payloads/<name>/<version>.tfs.sha256          # trust anchor
payloads/<name>/<version>.manifest.yaml       # manifest
mirror (embedded wins; else synthesized LOUDLY)
shims/                                        # spec 07
registries/<sha>.yaml (+.fetched-at)          # dispatch-
time registry cache (24 h TTL, spec 07; §4's stale-serve)
config.yaml                                   # spec 07
(YAML — never JSON)
keys/                                         # press-local
signing keys (spec 09)
keyring/trusted.pgp                           # the trusted
signer keyring (`tebako keys import`; spec 09 §2)
trust/<FINGERPRINT>.pub                       # add-registry
TOFU pins, read-only (spec 09 §9.1)
trust/ca/<name>.pem                           # the operator
CA store, read-only (spec 22 §4 Rule R4)
```


© 2026 The Tebako Project

The interpreter/image file names in a cache entry keep the index entry's `filename` / `image.filename` spellings verbatim (§2 — on windows the exe is suffix-less; the loader execs by full path and CreateProcess needs no `.exe`). The diagram shows the post-tebako#716 spelling; entries installed from the immutable ≤ v0.16.32 lines keep their `<ver>-<lv>-<triplet>` spellings forever — the two eras coexist in one store, and the directory name (`<lang>-<lv>-<ver>-<triplet>`) is era-free. The `[.exe]` diagram notation marks the synthesized fallback spelling, used only when no index entry is available (fat-payload installs, pre-identity manifests).

3.0a The manifest mirrors and the extension-slice scan

Every installed payload carries its L1 manifest beside the image as `<version>.manifest.yaml` (the embedded manifest transcribed at install; a legacy image without one gets a LOUDLY synthesized minimal mirror — identity only, no `augments`). The mirrors are the dispatcher's store-local reading of the extension-slice graph (spec 07 §2 step 3a): the scan for slices augmenting the dispatched payload reads ONLY these files — never the network, never the registry caches (trust was already enforced at install; a run re-verifies nothing). A corrupt or unparseable mirror is a loud skip of that one payload (`event=slice-skip reason=manifest-unreadable`), never a dispatch failure.

3.1 Home discovery

The store root resolves by precedence (single owner: `tpkg::runtime_store::tebako_home`; every component — CLI, shim, bootstrap, driver, journal — reads through it):

1. `$TEBAKO_HOME` (non-empty) — the explicit override.
2. The bundle-sibling tier — the process image sits inside a tebako store tree, making that tree the home. Three seats:
   - `<root>/<seat>/<tool>` beside a grammar-carrying `<root>/home` (the bundle layout's `bin/`; the seat name is irrelevant — the grammar is the witness, so `/usr/local/bin` never false-triggers);
   - `<home>/shims/<tool>` (the windows shim copies; a unix shim is a symlink, resolves to the real binary, and takes the first seat or no seat instead);
   - `<home>/runtimes/<entry>/<exe>` (a store runtime — the driver's seat).

     The store-grammar markers are: any of `config.yaml`, `shims/`, `payloads/`, `runtimes/`, `registries/`, `keys/`, `trust/`. A directory carrying none of them is not a home and the tier does not fire.
3. The platform default — `~/.tebako`; windows: `%LOCALAPPDATA%\tebako` (falling back to `%USERPROFILE%\.tebako`).

The tier is identity-preserving for managed installs: every managed seat resolves to the home the default tier would pick anyway (or does not fire), so the tier only ever lights up for a foreign tree — a bundle (spec 16 §7's MSI/pkg `bin/` + `home/` layout). When the bundle tier wins, the shim and the bootstrap export `TEBAKO_HOME` early so the whole

© 2026 The Tebako Project

process tree — dispatch execs, the runtime driver, spawned grandchildren — agrees on the one home.

A bundle home may be **read-only** (installed under `Program Files` / `/opt`): runs work (mounts never write), mutating verbs fail with a named IO error naming the unwritable path, and the audit journal degrades to best-effort silence.

4. Install and trust rules (locked)

- Per-entry **flock** (120 s timeout with stale-lock hint); install is tmp + rename — a partial install is invisible.
- **Commit order (locked 2026-09-16):** an install lands in three stages — the image + trust anchor stage first (dispatch-invisible bytes), the payload's `requires:` closure walks second, and the **manifest mirror's save is the commit point**. A version whose mirror is absent was never installed, whatever else the record holds: consumers keying on "installed" (the closure's cached-pin short-circuit included) test the mirror, and an aborted attempt's staged bytes re-run the tail from the cache hit — never a re-download, never a half-installed record masquerading as standing.
- The `.sha256` marker IS the trust anchor: present ⇒ the artifact was sha256-verified at install; re-verify only on re-fetch, **never per run**.
- Image-era cache entries hold artifacts only — no extracted `layout/` tree; press seeds its environment by extracting in-process through the TFS ABI and rebuilds per press.
- `TEBAKO_OFFLINE=1` — cache hit or hard error.
- **Stale-registry serve on fetch failure (locked 2026-09-12, roadmap 86):** the dispatch-time registry cache (`registries/<sha>.yaml` + `.fetched-at`, 24 h TTL — spec 07) degrades LOUD, never hard: cache PRESENT but stale + the refresh fetch failed → the stale cache SERVES, with a stderr warning and a journal line (`event=stale-registry-serve`, naming the ref and the fetch error); cache ABSENT + fetch fails → the named resolution error, unchanged; a fresh cache and a successful refresh are unchanged. `TEBAKO_OFFLINE=1` stays cache-or-named-error, unchanged. This is NOT a silent fallback: the registry is an index — the trust anchor is the artifact's `.sha256` / `.asc` verified at fetch time, never the registry's freshness, and a stale index can name only artifacts whose own anchors still verify at install.
- `tebako cache list` (runtimes and payloads) / `cache prune [--runtimes] [--payloads] [--all | --older-than Nd]` manage the cache. Bare `cache prune` touches runtimes only; `--payloads` adds the payload arm, which NEVER removes a pinned version — the protected set (config `defaults:` pins through the spec 07 §2 toolpin grammar, exact `payload@version` disable selectors, and each name's newest installed version, the roll-forward floor) binds even under `--all`: prune never strands a pin. Per-directory `.tebako-tools.yaml` pins are dispatch-time state and invisible to prune (a reachable one prints the caveat once). Removal deletes the version's whole

© 2026 The Tebako Project

record — the `.tfs`, its `.sha256` / `.origin` markers, the `.manifest.yaml` mirror, a materialized `.tree/`, the install lock — and the name dir when it goes empty.

- A run is a run: executing a package NEVER installs its payload slices. `tebako install <path>` installs a local package's payload slices explicitly — trailer slot digests are the anchors for signed (v2) packages, the computed digest for unsigned ones (the run's own enforcement strength, never upgraded); the runtime slot is NOT store-installed (it resolves into the runtime cache on first run, as always). Idempotent same-sha skip; same name+version with different content is never overwritten (loud warning + journal). Shims link only via the explicit `--shims` (spec 07). `TPKG_FLAG_NO_INSTALL` packages refuse (spec 02 §5).
- Lazy seeding (the scoped exception, locked 2026-08-25, tebako#460): a package that CARRIES artifacts (the self-contained preset — spec 23 §13) MAY seed the machine cache with them at run time: the runtime pair (exe + env image) and each carried payload slice, so later packages share them. The exception is exactly this wide and no wider:
  - best-effort — a seed that cannot complete NEVER blocks or fails the run; the carried artifacts keep serving from the package;
  - tmp + rename under the entry lock — a partial seed is invisible and the next run re-seeds;
  - idempotent — a same-sha entry skips;
  - trust anchors exactly as `tebako install`: trailer slot digests for signed (v2) packages, the computed digest for unsigned ones — the run's own enforcement strength, never upgraded;
  - journaled (`event=lazy-seed`), and shown by `tebako cache list` with the seeding package as origin;
  - `TEBAKO_OFFLINE=1` does not block seeding (no network is involved);
  - `TPKG_FLAG_NO_INSTALL` packages never seed (spec 02 §5).

    The shared-runtime preset needs no exception: its artifacts arrive through the ordinary resolve-download-verify-install path.

4a. Run-time payload resolution + lazy seeding (tebako#460, locked 2026-08-25 — the fat/lean unification)

One pipeline, one lock. `press` resolves the full composition closure at build time — the runtime_ref plus every payload ref, transitively — and locks concrete versions + sha256 digests into the L2 package manifest. What press tested is what runs, always.

© 2026 The Tebako Project

The embed dial. `-m lean|fat` is sugar over an embed set (`--embed=all| none|name,...`): `fat` embeds everything (runtime exe + env image + all payload slices, per #458's two-slot shape); lean embeds a subset. Embedding changes byte placement only — non-embedded slices resolve at run time **by the locked digests**, never by fresh semver resolution, fail-closed on mismatch.

Run-time payload resolution mirrors the runtime's own chain exactly: registry fetch → sha256 verify → tmp+rename into `~/.tebako/payloads/` → co-mount at the declared points. Transitive DEPENDS walk with cycle detection; `TEBAKO_OFFLINE=1` is cache-or-named-error. Named errors: an unresolvable slice is the 75 class; a digest mismatch is the trust class; a cycle is 65.

Lazy seeding (the one exception to §4's run-is-a-run). A fat package's carried slices are byte-identical with registry artifacts; at first run the loader MAY copy absent slice regions into the machine cache (tmp+rename, the trailer-slot digest the trust anchor for signed packages / the computed digest for unsigned; journaled; idempotent same-sha skip). Strictly best-effort: never blocks, never fails, invisible when partial. The runtime pair seeds the runtime cache identically. Net: a fat download is a cache prime with a runnable side effect; fat and lean converge byte-for-byte.

Trust uniform: embedded or resolved, the same sha256 anchors, the same unsigned-is-loud + journal rules, the same `TEBAKO_REQUIRE_SIGNED=1` fail-closed, verification at fetch/install — never per run.

5. Resolution chains

Runtime for a payload (dispatch time): payload's declared requirement (engine + optional implementation + constraint) → newest COMPATIBLE runtime already cached (no download) → else download the newest compatible from the runtime releases → verify → cache. Swapping runtimes never touches the payload. **The min-runtime floor (spec 03 §2.9, schema_minor 15, tebako#666):** when the payload manifest declares `min_runtime_tebako`, cached runtimes whose own tebako version is below the floor are discarded BEFORE the compatible pick (they are named in the miss errors, never silently preferred) and the download path runs instead; a downloaded runtime below the floor (a pinned stale line) is the named stale-runtime refusal. `TEBAKO_OFFLINE=1` stays cache-or-named-error with the floor named. The floor gates every runtime the dispatch resolves — the entrypoint's own and the spawned edges' (spec 30/32); a provider payload's floor gates the provider's pair.

Runtime for a shared-runtime package (first run): trailer runtime_ref → cache hit → use; miss → download from the index (per-package shard primary, manifest.json next, SHA256SUMS fallback — §2's order) → sha256-verify → flock'd atomic install. (Preset names per spec 23 §13.2: *shared-runtime* is the default preset and the successor of the deprecated name *lean*; *self-contained* succeeds *fat*.) Self-contained: the runtime pair is carried as two package slots — the interpreter exe and the env image (spec 19 §6.1): the

© 2026 The Tebako Project

exe stages into the runtime cache, the env image mounts from the package file through the ordinary `<self>:<slot>:<mount>` grammar, both verified against the trailer slot digests, and both seed the cache per §4's lazy-seed rule. (Pre-#458 packages carry the runtime as one legacy `format_id = 4` slot, extracted and verified against `;sha256=` — readers keep serving that form; presses never emit it.)

Compatibility model: pure-language payloads take a range (`>= 3.3, < 5.0`) — any newer runtime works; native-extension payloads lock to the ABI line they were built against, which is TWO-dimensional: the version line (`~> 3.3.0`) AND the platform line (`abi: arm64-darwin-23` — the runtime's own platform string; ruby: `Gem::Platform.local.to_s`, from the staging runtime's `RbConfig::CONFIG["arch"]`). A newer line needs a new payload build (or an SDK rebuild path); a wrong line on EITHER axis produces a named compatibility error, never a segfault. Runtimes publish their platform string as the per-package `abi` key in the release index (spec 13); cached runtimes installed before the field existed stay eligible (the compat window — a payload's abi check never fails against an unknown line).

Version ordering (the plain-wins rule — the single owner is `tpkg::versions::compare`; every consumer flows it). Versions are dot-separated components, missing components zero. Within one component: a leading numeric prefix compares numerically (`1.10 > 1.9`); at an equal prefix a PLAIN component ranks ABOVE a suffixed one (`3.13.15 > 3.13.15-jit` — semver's release > prerelease rule), so a factory's build variant (a `-jit`, `-debug`, … line) never silently outranks its plain twin in a newest-satisfying pick; two suffixes order lexicographically; a component without a numeric prefix falls back to whole-component string order. Constraint MATCHING is unaffected: a variant version still satisfies an open constraint, so a platform where ONLY the variant exists still resolves — the variant simply never wins a max pick against the plain twin. A variant is SELECTED through the pin surface (the config `version:` pin, the registry default), which names versions exactly and never compares.

Variant suffixes in constraints (payload-manifest schema_minor 12). The constraint grammar (spec 03) admits the SAME suffixed component shape versions carry — each dot-separated component is leading decimals plus an optional `-label` (`[0-9A-Za-z-]+`) — so an exact pin can NAME a variant line (`= 1.16.9-ruby4.0` — the spec 03 §2.8 binding rule against a line-split base, one slice per base line). Matching semantics (`tpkg::versions`, the single owner): equality clauses are suffix-exact — the variant and its plain twin are DIFFERENT versions (`= 1.16.9` never matches `1.16.9-ruby4.0`); ordering clauses follow `compare()` unchanged — a plain bound excludes variants ranking below it (`>= 1.16.9` does not match `1.16.9-ruby4.0`), a suffixed bound admits its own line upward (`>= 1.16.9-ruby4.0` matches the variant, the plain twin, and everything newer); `~>` computes its upper bound from the numeric core under the same component model as `compare()` — the label's own dot counts (`~> 1.16.9-ruby4.0` is a four-component spelling, so the bound is `< 1.16.10`; for `< 1.17` write the explicit range). Readers older than minor 12 reject a suffixed clause by name (exit 65) — fail-closed, never a misread pin.

The implementation axis (spec 28 §8): `engine` names the LANGUAGE — mri, jruby and truffleruby are all `engine: ruby`, told apart by the runtime's `provides.implementation`. A requirement WITHOUT `implementation` matches any cached runtime of the engine whose `language_version` satisfies the constraint (the pure-language case: one build, any implementation — jruby 9.4 declares `language_version:`

© 2026 The Tebako Project

"3.1", so a >= 3.3 requirement does not match it and a >= 3.1 one does). A requirement WITH `implementation` reads its constraint against that implementation's OWN version line (`{engine: ruby, implementation: jruby, constraint: "~> 9.4"}`) and matches no other implementation. An `abi:` in force requires `implementation` — ABI and platform lines are per-implementation by construction; a native requirement naming no implementation is a named manifest error (spec 03 §2.2). Compatibility never crosses implementations silently: the chosen runtime's implementation is journaled with the resolution event.

The python wheel-tag mapping (`engine: python` — locked 2026-09-08). Python native extensions carry CPython's own two-tag compatibility grammar (the PEP 425/600 wheel tags). Tebako does not invent a second grammar — it maps that one onto the two-dimensional ABI line above, once, here; the resolver and the factory REFERENCE this table and never re-derive it in code comments.

- **Version line.** The wheel ABI tag `cp3X` (`cp311`, `cp312`, ...) IS the version line `~> 3.X.0`: a native-extension payload baked against CPython 3.X declares `runtime_requirement: {engine: python, constraint: "~> 3.X.0", abi: <the built-against runtime's abi>}`. Pure-python payloads take the range form exactly as ruby's pure-language case (xml2rfc's `requires_python >= 3.10` is the first instance).
- **The stable ABI (`abi3`) is a bake-time concern, never a resolver concern (locked).** An upstream `cp3Y-abi3` wheel is compatible with every CPython line `>= 3.Y`; the factory consumes that width by baking the abi3 wheel into EACH targeted (line × triplet) payload variant. The baked variant records the TARGET runtime's line and `abi` exactly like a line-specific build — the resolver's grammar gains no `abi3` case and the compatibility width never leaks into the wire.
- **Platform line.** The python runtime's published `abi` key (spec 13) is the CPython extension-suffix stem (sysconfig `EXT_SUFFIX` without the leading dot), e.g. `cpython-312-x86_64-linux-gnu.so`. The payload records the built-against runtime's `abi` verbatim; comparison is the exact-string rule above, unchanged. macOS's suffix is arch-neutral (`cpython-312-darwin.so`) — the arch axis rides the triplet alone there; no second arch axis is ever introduced.
- **Wheel platform tags ↔ triplets** — the one table:

TABLE 1

wheel platform tag (PEP 425/600)	tebako triplet
`manylinux_*_x86_64`	`linux-gnu-x86_64`
`manylinux_*_aarch64`	`linux-gnu-arm64`
`musllinux_*_x86_64`	`linux-musl-x86_64`
`musllinux_*_aarch64`	`linux-musl-arm64`
`macosx_*_x86_64`	`macos-x86_64`

© 2026 The Tebako Project

wheel platform tag (PEP 425/600)	tebako triplet
`macosx_*_arm64`	`macos-arm64`
`macosx_*_universal2`	split per-arch at bake time — the payload stays per-triplet
`win_amd64` / `win_arm64`	RESERVED — the windows python runtime is not yet shipped; readers MUST NOT guess a mapping

The musllinux-vs-manylinux (libc) distinction rides the triplet axis ONLY (`linux-musl` vs `linux-gnu`) — no wheel-tag axis, no libc key, no ad-hoc second axis anywhere in the grammar. A tag's policy version (`manylinux_2_28`, `musllinux_1_2`, ...) is a build-floor statement consumed at bake time, never a resolution axis: a runtime built against any floor serves its triplet.

- Wrong line = the named compatibility error on either axis, unchanged (never a segfault): the python instance names the selector and lists the available cached variants per spec 28 §4/§10.

6. The fetch pipeline (parallel pulls, streaming integrity — locked 2026-09-18)

Every dynamic pull of ARTIFACT bytes (runtime exe + env image + dll, payload slices, detached `.asc` signature assets, trust-anchor keys — spec 09 §10) runs through the one fetch pipeline (`tebako_resolve::plan`), never through ad-hoc per-caller download loops. Release INDEX reads (shards, manifest monoliths, SHA256SUMS, registry YAML) stay small buffered GETs; the pipeline is for artifacts.

- The plan. Resolution produces the whole closure up front; the pipeline takes it as an explicit `FetchPlan`: a subject line (the progress header's `<what>`) plus one item per artifact carrying `{reference, sha256 pin, size hint, tmp dir, commit closure}`. The commit closure owns the artifact's install semantics (signature verification, the store's lock + rename + markers) — the pipeline owns transport, integrity, and scheduling; neither crosses over.
- Parallelism. A std::thread worker pool — `jobs` from `TEBAKO_FETCH_JOBS` (env) over `fetch_jobs` (`~/.tebako/config.yaml`) over the default 3 (docker's number); 1 is sequential (the debug shape). An unparseable or zero value is a NAMED error, never a silent clamp. No async runtime, no new dependencies (spec 14).
- Streaming integrity (one pass, constant memory). Each worker streams its response body into the item's tmp file WHILE updating a streaming sha256 hasher; the pin

© 2026 The Tebako Project

compares at end-of-stream. The pre-pipeline shape (buffer the whole body into a Vec<u8>, hash as a second pass — a 200 MB image pinning 200 MB of RAM) is RETIRED: artifact bytes never materialize whole in memory on the download path. A signature verification that needs the bytes reads the staged tmp file once, after ITS download completes — overlapped with the plan's remaining streams — and never after the rename.

- **Retry discipline, per worker per connection.** The existing throttle/backoff law is unchanged (the tebako-http SSOT: Retry-After honored exactly, the 60 s × 2^n hintless exponential, THROTTLE_ROUNDS rounds; DOWNLOAD_ATTEMPTS for transient failures) — it now runs per worker. A mid-stream failure retries the artifact FROM ZERO (tmp truncated, hasher reset): a resumed partial body would fail the pin anyway.
- **Failure law.** Any artifact's transport/hash/signature/commit failure CANCELS THE PLAN: in-flight streams abort at the next chunk, queued items never start, the pool joins, every tmp file of the plan is dropped, and the FIRST named error surfaces (its exit code unchanged — the taxonomy of spec 06 §4 / the caller's table is untouched). A partial install never appears in the store: tmp+rename (§4) already makes the bytes invisible, and the cancel path removes the tmp files — proven by test, not asserted.
- **The store is concurrency-safe by design** (§4's per-entry flock + content-addressed byte-identical artifacts): two workers never share an entry — one plan holds disjoint store slots, and a foreign process installing the same entry serializes on the same flock with the same 120 s stale-lock-hint timeout. Parallel fetch changes nothing there (test-proven).
- `TEBAKO_OFFLINE=1` keeps its cache-or-named-error semantics, evaluated per item before its stream starts; `TEBAKO_REQUIRE_SIGNED=1` gates exactly where it gates today (the commit closure's signature policy is the caller's, unchanged in meaning).
- **Rendering:** spec 06 §5a — the pipeline renders through tebako-term v2 and nothing else prints download progress.

© 2026 The Tebako Project